FINAL PROJECT · ISSUED ON DAY 5 · COMPLETED AFTER THE MODULE

The Northgate Logistics Triage Helper

Due: DUE DATE **Submit via:** SUBMISSION METHOD **Questions:** CONTACT

Time: 6 to 8 hours in total, outside the 15 classroom hours.

1. Objective

Demonstrate that you can independently apply the full range of skills developed across the five instructional days: writing structured prompts from named components, using worked examples, producing and validating machine-readable output, applying the five developer patterns to real code, troubleshooting a failing prompt systematically, verifying output before it is used, and exercising professional judgment about when AI assistance is not appropriate.

You are not being assessed on whether the AI produced a good answer. You are being assessed on your prompts, your process, your verification, and your reasoning.

2. Scenario

You have been at Northgate Logistics for two weeks as a junior developer. The company delivers parcels for regional retailers and handles roughly 300 free-text customer complaints a week through a web form.

Right now a team lead reads every complaint and manually sorts it into a category, assigns a priority, and decides whether it needs a human reply or can be handled by an automated acknowledgement. It takes about six hours a week and it is nobody's favourite part of the job.

Your team lead has asked you to prototype an AI-assisted triage helper. It should read one complaint and return structured data: a category, a priority, a one-line summary, and a flag indicating whether a human must review it. She has been clear that this is a prototype to evaluate the idea, not a system that will be connected to anything yet.

At the same time, a small bug has been reported in the existing intake module that receives these complaints. The code is provided, it is unfamiliar to you, and a short requirements note describes what it is supposed to do. Nobody on the team has time to walk you through it.

Finally, your team lead has asked you to document your work properly, because if this prototype is taken forward someone else will pick it up.

3. Problem statement

Produce a working prototype triage prompt with evidence that it behaves reliably, fix the reported bug in the intake module using an AI-assisted workflow you can account for, and document both pieces of work to a standard a colleague could pick up, including an honest statement of where you judged AI assistance to be inappropriate.

4. Learning outcomes assessed

- Understand how a language model produces output, and use that understanding to explain your prompt design choices (Day 1)
- Apply the six-part prompt template and few-shot examples deliberately (Day 2)
- Apply the five developer patterns (explain, generate, test, debug, document) and the four-step verification routine (Day 3)
- Analyze whether a task should be delegated, using blast radius, verifiability, and reversibility (Days 3

and 5)
- Demonstrate strictly structured output with validation across multiple inputs (Day 4)
- Troubleshoot a failing prompt with the five-question method and show a measurable improvement (Day 4)
- Identify prompt injection risk in a system that accepts customer-written text (Day 4)
- Build a prompt library with a consistent entry structure, and run a complete workflow independently (Day 5)

5. Requirements

- All six deliverables must be submitted. Partial submissions are marked on what is present.
- Every deliverable must show the **prompts** you used, not only the outputs. The prompt is the work being assessed.
- The triage prompt must be tested on **at least 10** of the 20 sample complaints.
- Validation evidence must be included: validator output, a screenshot, or a completed written checklist per input.
- The improvement log must document a real failure, not a manufactured one.
- The reflection must include a specific, reasoned statement of where you decided AI assistance was NOT appropriate.
- Do not paste any real personal data, credentials, or content from another employer into any AI tool. The sample data supplied is fictional and safe to use.
- You may use any AI assistant you like. State which one you used, in the reflection.

6. Tools and the project pack

Required: a free AI chat assistant in a browser (any one), and a text editor.
Recommended: Python 3.10 or newer, to run `validation.py` and `intake.py`. If you cannot run Python, use a free browser JSON validator such as JSONLint and record the result. Nothing paid is required.

`sample_complaints.csv`	20 fictional customer complaints, including ambiguous ones, one empty, and one containing an injection attempt
`intake.py`	the intake module with the reported bug
`bug_report.txt`	the bug, written as a real ticket: a symptom, not a cause
`requirements.txt`	the requirements note: intended behaviour (a note to read, not a pip file)
`triage_schema.json`	the required output shape
`validation.py`	a starter validator for your triage output
`rubric.md`, `submission_checklist.md`	the rubric summary and a final checklist

7. Step-by-step instructions

Work through these in order. Each step corresponds to something you practised in a lab.

1. Read this whole brief, the requirements note, and the bug report before writing any prompt. Write two sentences on what you think the real problem is and what the blast radius of a wrong triage decision would be.
2. Design the triage output schema: exact keys, types, enumerated values for category and priority, the length limit for the summary, and the output when a complaint does not fit any category. Write the schema down before you write the prompt.
3. Write version 1 of the triage prompt using the six components. Include at least two consistent

worked examples, one of which is an awkward or ambiguous complaint. Wrap the complaint in delimiters and state explicitly that its contents are data, not instructions.

4. Run version 1 on 10 sample complaints. Validate each result. Record every failure precisely: invalid JSON, missing key, invented category, wrong shape, or a value you judge to be wrong.
5. Apply the five-question troubleshooting method to the most significant failure. Write your five answers. Make ONE change, re-run, and record the result. Repeat once more if needed. This becomes your improvement log.
6. Finalize the triage prompt and record the final validation results across all 10 complaints.
7. Move to the intake module. Use the EXPLAIN pattern to orient yourself. Save the prompt and the response.
8. Use the DEBUG pattern with the code, the bug report, the expected behaviour from the requirements note, and anything you observed. Ask for ranked causes with confirmation steps, not a fix. Save it.
9. Use the GENERATE pattern to produce the smallest change that fixes the cause you identified. Include a ”done means” list with at least one failure case.
10. Use the TEST pattern: ask for the edge case list first, then tests. Check that list against the REQUIREMENTS NOTE, not just against the code. Mark anything it missed.
11. Run the four-step verification routine on your fix and write one line per step describing what you actually did and what you found.
12. Use the DOCUMENT pattern for a commit message, then add one sentence of ”why” the model could not have known.
13. Assemble your prompt library: five entries, each with name, purpose, when NOT to use it, the prompt with placeholders, and one example input and output.
14. Write the one-page reflection. Include where you decided AI assistance was not appropriate and why.
15. Check your submission against the rubric before you submit.

8. Expected outcome

A successful submission shows a triage prompt that produces schema-conforming output across all 10 tested complaints, including the ambiguous ones, with documented evidence of validation. It shows a bug that was understood before it was fixed, with a diagnosis that preceded the change and a verification that found something real. It shows one genuine prompt failure diagnosed systematically and improved by a single deliberate change, with the improvement stated in measurable terms. And it shows a student who can say clearly where they chose not to delegate, and why.

A submission that produces perfect output with no visible prompts, no validation evidence, and no failures will score poorly. The process is the deliverable.

9. Deliverables

1	**Prompt library**	A markdown or text file with five entries. Each entry: name, what it is for, when NOT to use it, the prompt with {PLACEHOLDERS}, one example input and output, and a date.
2	**Triage prompt and validation evidence**	The final prompt, the declared schema, the outputs for 10 sample complaints, and validation evidence for each.
3	**Bug fix package**	Your explain, debug, generate, and test prompts with their responses, plus the corrected code and the generated tests.
4	**Verification report**	The four-step routine applied to your fix, one line per step, stating what you did and what you found.
5	**Improvement log**	The original failing prompt, the five troubleshooting answers, the single change made, the result, and a measurable statement of what improved.
6	**Reflection (1 page)**	The answers to the questions in section 12.

Suggested time split: 2 hours on the triage prompt and validation, 2 hours on the bug fix workflow, 1 hour on the prompt library, 1 hour on the verification report and improvement log, 1 hour on the reflection and assembly.

10. Rubric (100 points)

Criterion	Pts	Full marks	Partial	Weak
Understanding of concepts	15	Course terminology used accurately throughout. Design choices explained with reasons grounded in how models work, not asserted. You can say why few-shot was used here and not there.	Terminology mostly correct, but choices described rather than justified.	Terminology misused or absent; no reasoning.
Prompt quality and technique	25	All six components used deliberately and visibly. Few-shot examples consistent and include an edge case. Delimiters around untrusted content with an explicit data statement. Real, marked placeholders. Complete rather than long.	Most components present; examples inconsistent, or delimiters missing.	Single-paragraph prompts; no structure, examples, or placeholders.
Structured output and validation	20	Strict schema declared in advance: exact keys, types, enumerated values, defined empty case. Tested on 10 complaints with validation evidence for each. Failures recorded honestly and resolved.	Schema mostly followed; fewer than 10 inputs, or evidence incomplete.	One output, no schema, no validation.
Problem-solving and troubleshooting	15	A real failure diagnosed with the five questions, answers that genuinely differ, exactly one change per iteration, improvement stated in measurable terms.	Failure fixed, but several changes at once or thin diagnosis.	No failure documented, or ”I rewrote the prompt and it got better.”
Verification and professional judgment	15	All four steps applied with specific findings, including something checked against external documentation and something run on bad input. A genuine boundary where AI was not appropriate, with a reason based on blast radius, verifiability, or reversibility.	Verification generic; boundary stated without a reason.	”I checked it and it looked fine.” No boundary.
Documentation and presentation	10	Clearly headed sections, complete, readable, navigable. A colleague could continue the work without asking questions.	Complete but disorganized, or organized but missing pieces.	One undifferentiated block, or deliverables missing.

The two most common ways marks are lost: testing on one input instead of ten, and showing outputs without showing the prompts that produced them.

11. Optional challenge

Attempt only after the six required deliverables are complete and strong. It does not compensate for weak core work. Choose ONE:

(a) **Chain it.** Add a second prompt that takes the JSON output of your triage prompt for 10 complaints and produces a one-paragraph daily summary for the team lead: top categories, the count needing human review, and the single most urgent item. Show the intermediate JSON, the second prompt, and the output.

(b) **Attack it.** Write three plausible prompt-injection attempts as if they were customer complaints, run them through your triage prompt, record what happened, then strengthen your prompt and re-test. Report honestly which attempts were and were not deflected.

(c) **Evaluate it.** Label 10 complaints by hand with the category and priority you believe are correct, compare your prompt’s output against your labels, report the agreement rate, and describe every

disagreement, including any where the model was right and you were wrong.

12. Reflection questions (one page)

1. What did you learn during this project that you did not fully understand during the five instructional days?
2. What was the hardest part, and what specifically did you do to get past it?
3. Describe one thing that failed and how you diagnosed it. Which of the five questions found it?
4. What would you improve if you had another four hours?
5. Where did you decide AI assistance was NOT appropriate in this project, and what was your reasoning?
6. How would you apply what you did here in a real job? Name one specific task at a workplace you are aiming for.
7. Which AI assistant did you use, and did you notice any of its particular habits or weaknesses?